

Relatório de validação — 2026-10-08

Entregável 6 do escopo (context/06 §5). Gerado por `python -m analysis.gerar_relatorio_validacao` com `--banco`; nenhum número foi digitado à mão.

1. Identificação

Campo	Valor
Data da geração	2026-10-08
Código que gerou o relatório (<code>git rev-parse HEAD</code>)	13ab59fdcd9804a09f06e6958b871b494d5a2420
Código das 650 execuções do Bloco 8 (<code>execucoes.csv</code>)	85b1803
Código da ponte nas 100 passagens de H3	5409d93
Versão do SUMO (na geração)	Eclipse SUMO sumo 1.27.1

O lote não grava a versão do SUMO. A linha acima é a do SUMO instalado quando o relatório foi gerado, e só vale para o Bloco 8 se ele não foi trocado depois (docs/plano-desenvolvimento.md, Bloco 0, registra a instalação da 1.27.1).

Hash dos parâmetros

sha256 de cada arquivo como está no commit dos dados (85b1803), e se continua igual no HEAD:

Arquivo	sha256 (16 primeiros)	No HEAD	Commits que o mudaram depois
backend/config/parametros.yaml	a670233c0f09d3cb	igual	04aa3a4 feat(simulacoes): faixas de seed reservadas, com o uso de cada uma, no dashboard
backend/config/parametros.hardware.yaml	fb808874a2244001	igual	
backend/config/politica_desempate.yaml	b7e8b3acfedab2bd	igual	
sim/config/cenarios.yaml	dac549735effb8af	mudou	
sim/config/mapa_fases.yaml	53a47c75566c9bff	igual	
sim/demanda/veiculos.typ.xml	8bf31d3836f9155e	igual	
sim/rede/malha.nod.xml	59af09595b7c0b0e	igual	
sim/rede/malha.edg.xml	97a62be207696137	igual	
sim/rede/malha.con.xml	84cda25925f6674c	igual	
sim/rede/malha.tll.xml	0a5017c5b66a1a80	igual	
sim/rede/malha.typ.xml	f4f9ea8e8abd2eed	igual	

No banco, `execucao_simulacao.parametros` das 650 execuções de 85b1803: 1 snapshot(s) distinto(s) — 13b6efcc8b392f43 em 650 (sha256 do JSON com chaves ordenadas).

2. Ambiente de teste

A máquina e as ferramentas onde o relatório e as suítes rodaram. O log do lote não registra a máquina; o lote rodou num worktree no mesmo notebook (09, “Bloco 8 rodado”).

Item	Versão / valor
Sistema operacional	Windows-11-10.0.26200-SP0
Processador	AMD64 Family 25 Model 33 Stepping 2, AuthenticAMD
Núcleos lógicos	16
Memória	31.9 GiB
Python	3.12.10
SUMO	Eclipse SUMO sumo 1.27.1
Node.js	v24.19.0
Docker	Docker version 29.7.2, build a7dcaa6
arduino-cli	arduino-cli Version: 1.5.1 Commit: 01f3d4f2b Date: 2026-06-05T10:22:12Z
Núcleo Arduino arduino:avr	1.8.8
Núcleo Arduino esp8266:esp8266	3.1.2
fastapi	0.115.14
uvicorn	0.52.4
sqlalchemy	2.0.52
alembic	1.19.1
psycpg	3.3.4
pydantic	2.13.4
pyjwt	2.15.1
httpx	0.28.1
pyyaml	6.0.3
structlog	26.1.0
pyserial	3.5
numpy	2.5.3
scipy	1.18.1
pandas	3.0.6
matplotlib	3.11.2
pytest	9.1.1
hypothesis	6.165.10
testcontainers	4.15.0
ruff	0.16.4
mypy	2.3.1

3. Matriz de execuções

- Planejadas (primeira linha de `lote_bloco8.log`): 650
- Válidas (`execucoes.csv`): 650
- Seeds: 1..50 (50)
- Registros em `descartes.csv`: 36 (REEXECUCAO: 36)

Cenário	Modo	Execuções válidas
intenso	FIXO	50
intenso	PREEMPCAO	50
intenso	PREEMPCAO_COMPENSADA	50
leve	FIXO	50
leve	PREEMPCAO	50
leve	PREEMPCAO_COMPENSADA	50
moderado	FIXO	50
moderado	PREEMPCAO	50
moderado	PREEMPCAO_COMPENSADA	50
multiplas_emergencias	FIXO	50
multiplas_emergencias	PREEMPCAO	50
multiplas_emergencias	PREEMPCAO_COMPENSADA	50
multiplas_emergencias	PREEMPCAO_ML	50

Motivo de cada registro de `descartes.csv`:

Tipo	Motivo	Registros
REEXECUCAO	lote interrompido pelo reinício do computador em 2026-10-07 (28 de 650 concluídas); reexecução do zero	36

REEXECUCAO não é execução reprovada: é a remoção, registrada, das linhas que uma corrida interrompida deixou no banco, antes de rodar tudo de novo (06 §4). Reprovadas por `validar_execucao()` (DESCARTE ou FALHA): 0.

4. Rastreabilidade requisito → teste → resultado

A tabela de `context/06 §2`, lida do arquivo, com o resultado de cada arquivo de teste nas suítes (seção 4.1) e, onde 06 §2 aponta a bancada ou o Bloco 8, o dado gravado julgado pelo critério declarado. Um requisito é PASSOU só se tudo o que o sustenta passou; uma falha esperada (`xfail`) continua sendo FALHOU.

Req.	Caso de teste	Testes	Evidência gravada	Resultado
RF01	VE a 480 m na rota é detectado; a 520 m não é	<code>test_deteccao.py</code> : 20/20 passaram	—	PASSOU
RF01	VE a 100 m em linha reta mas fora da rota não é detectado	<code>test_deteccao.py</code> : 20/20 passaram	—	PASSOU
RF02	Da detecção ao início da atuação < 3 s (decisão P14)	<code>test_e2e_preempcao.py</code> : 6/6 passaram	Bancada, item 7: maior latência das 100 passagens de H3 = 46,3 ms (limite 3.000 ms)	PASSOU

Req.	Caso de teste	Testes	Evidência gravada	Resultado
RF03	VE atravessa 8 cruzamentos sem parada	<code>test_corredor_verde.py</code> : 2/3 passaram, 1 xfail	Bloco 8, braço PREEMPCAO, VEs sem nenhuma parada: intenso 198/250 (79,2%; espera mediana de quem parou 9,2 s, no FIXO 68,5 s); leve 185/298 (62,1%; espera mediana de quem parou 2,3 s, no FIXO 109,5 s); moderado 210/262 (80,2%; espera mediana de quem parou 2,3 s, no FIXO 86,2 s); multiplas_emergencias 289/566 (51,1%; espera mediana de quem parou 4,6 s, no FIXO 58,7 s)	FALHOU (falha esperada, xfail)
RF04	WebSocket emite mudança de estado em < 500 ms	<code>test_ws.py</code> : 3/3 passaram; <code>test_bancada_integrada.py</code> : 4/4 passaram; <code>frontend/src/stream/*.test.ts(x)</code> : 24/24 passaram	—	PASSOU
RF05	Toda preempção gera linha em <code>log_prioridade</code>	<code>test_persistencia.py</code> : 6/6 passaram; <code>test_bancada_integrada.py</code> : 4/4 passaram	—	PASSOU
RF06	Posição do VE é publicada a ≥ 1 Hz	<code>test_ws.py</code> : 3/3 passaram; <code>frontend/src/stream/*.test.ts(x)</code> : 24/24 passaram	—	PASSOU
RF07	Mudança de rota do VE recalcula os TLS-alvo	<code>test_recalculo.py</code> : 5/5 passaram	—	PASSOU
RNF01	p95 de <code>latencia_decisao_ms</code> < 100 ms em 10.000 chamadas	<code>test_desempenho.py</code> : 5/5 passaram	—	PASSOU
H3	p95 de <code>latencia_total_ms</code> (<code>t_deteccao</code> → <code>t_atuacao</code>) < 200 ms em 100 passagens de bancada	<code>test_e2e_preempcao.py</code> : 6/6 passaram	Bancada, item 7b: p95 = 31,6 ms em n = 100 (limite 200 ms)	PASSOU
RNF02	Sistema opera 60 min contínuos sem vazamento de memória	<code>test_soak.py</code> : 2/2 passaram; <code>test_uma_hora_de_operacao_sem_vazamento</code> gravou: <code>amostras_rss</code> = 154, <code>rss_referencia_mb</code> = 232.4, <code>rss_max_mb</code> = 232.4, <code>rss_min_mb</code> = 232.4, <code>decisoes</code> = 36000	Bancada, item 12: maior operação contínua sem reinício do UNO = 33,1 min (pedido: 30 min, relógio de parede)	PASSOU
RNF03	Motor processa malha de 32 TLS mantendo p95 < 100 ms	<code>test_desempenho.py</code> : 5/5 passaram	—	PASSOU
RNF04	POST sem X-Device-Token válido → 401; UID não cadastrado → 403	<code>backend/tests/api/test_seguranca.py</code> : 8/8 passaram	—	PASSOU

Req.	Caso de teste	Testes	Evidência gravada	Resultado
RNF04 (dashboard)	Login certo → token de 8 h; credencial errada → 401 sem dizer qual parte errou; cada escrita do operador sem token, com token vencido, falso ou malformado → 401; com token → passa; GETs e / <code>simulacoes/transmissao</code> abertos; sem login configurado → 503; <code>alg: none</code> recusado	<code>backend/tests/api/test_autenticacao.py</code> : 29/29 passaram; <code>backend/tests/test_autenticacao.py</code> : 17/17 passaram; <code>frontend/src/auth/sessao.test.tsx</code> : 4/4 passaram	—	PASSOU
RF01 (P20, API)	Tag reconhecida sem ocorrência → 200 <code>SEM_OCORRENCIA</code> ; com ocorrência → <code>PREEMPCAO_SOLICITADA</code> com a criticidade, e <code>id_correlacao</code> igual na detecção e no log; repetição em 2 s não grava	<code>backend/tests/api/test_dececoes.py</code> : 9/9 passaram	—	PASSOU
RNF05	Taxa de reconhecimento de tag $\geq 95\%$ em 100 leituras	—	Bancada, item 4: 100 de 100 leituras chegaram ao UNO com a rua certa (100,0%; mínimo 95,0%), emissor no USB	PASSOU
RNF07	<code>core/</code> não importa framework nem I/O	<code>test_arquitetura.py</code> : 50/50 passaram	—	PASSOU
RF01 (P20)	Tag reconhecida sem ocorrência ativa não é autorizada; com ocorrência, é, e carrega a criticidade	<code>test_autorizacao.py</code> : 4/4 passaram	—	PASSOU
RF01 (P20)	Na simulação, VE sem o parâmetro criticidade não chega ao motor	<code>sim/tests/test_adaptador.py</code> : 8/8 passaram	—	PASSOU
E8 (P20)	Criticidade vence tipo; no mesmo nível, a ordem antiga (tipo → ETA → em curso) é preservada	<code>test_conflito.py</code> : 17/17 passaram	—	PASSOU

Resumo: FALHOU: 1, PASSOU: 19.

4.1 Suítes

Arquivo	Comando	Rodou em	Resultado
<code>pytest_padrao.xml</code>	<code>python -m pytest</code> (suíte padrão, com os testes de banco)	2026-10-08T19:47:25.434162-03:00	1266/1267 passaram, 1 xfail
<code>pytest_sumo.xml</code>	<code>python -m pytest -m sumo</code>	2026-10-08T19:49:53.803231-03:00	27/27 passaram
<code>vitest.xml</code>	<code>npx vitest run</code> (frontend)	2026-10-08T22:52:26.691Z	74/74 passaram

Casos que falharam (inclusive os esperados):

Suíte	Caso	Desfecho
pytest_padrao.xml	tests.e2e.test_corredor_verde::test_nenhum_ve_para_nos_bracos_com_preempcao	XFAIL

5. Invariantes de segurança

Simulação: o `VerificadorSeguranca` em todo passo das execuções do Bloco 8 (violacoes de `execucoes.csv`; o lote não separa por invariante). Bancada: toda a telemetria gravada (`telemetria_bancada.csv`, todas as sessões), com `bridge.verificar.violacoes` e a folga de 60 ms do checklist. Testes: os casos dos arquivos que exercitam cada invariante, nas suítes da seção 4.1.

Invariante	Simulação (Bloco 8)	Bancada (telemetria)	Testes
I1	0 em 650 execuções (contagem conjunta de I1 a I5)	0 ST com verde nos dois eixos, de 9082	122/122 passaram
I2	0 em 650 execuções (contagem conjunta de I1 a I5)	0 em 1761 mudanças de luz	122/122 passaram
I3	0 em 650 execuções (contagem conjunta de I1 a I5)	0 em 1761 mudanças de luz	122/122 passaram
I4	0 em 650 execuções (contagem conjunta de I1 a I5)	0 em 1761 mudanças de luz	122/122 passaram
I5	0 em 650 execuções (contagem conjunta de I1 a I5)	maior vermelho contínuo de uma aproximação: 63,0 s (limite 120 s)	46/46 passaram
I6	não se aplica (I6 é do firmware, 01 §6)	2 de 2 TIMEOUT a 30 s com volta pelo eixo oposto (critério do item 11); 21 PREEMP_INI e 17 PREEMP_FIM na telemetria	99/99 passaram

Ainda na simulação: 0 colisões e 0 teleportes nas mesmas execuções.

6. Latências

Percentis pelo posto mais próximo. A simulação mede a latência de **decisão** (RNF01, `motor.avaliar()` cronometrado); a bancada, a latência **fim a fim** de H3 (da leitura da tag ao `PREEMP_INI`). São grandezas diferentes (decisão P2).

Ambiente	Grandeza	n	Mín (ms)	Média (ms)	p95 (ms)	p99 (ms)	Máx (ms)
SIMULA-CAO	decisão, PREEMPCAO (exemplares)	144000	0,006	0,042	0,110	0,158	4,457
SIMULA-CAO	decisão, PREEMPCAO_COMPENSADA (exemplares)	144000	0,006	0,051	0,116	0,155	12,163
SIMULA-CAO	decisão, PREEMPCAO_ML (exemplares)	36000	0,007	0,053	0,141	0,188	0,715
SIMULA-CAO	decisão, pior execução entre 450	—	—	0,091	0,199	0,295	12,163
HARD-WARE	fim a fim (H3), sessão das 100 passagens	100	0,1	25,7	31,6	43,3	46,3
HARD-WARE	fim a fim, sem carimbo atrasado (≥ 10 bytes)	98	23,4	26,1	33,0	46,3	46,3

Ambiente	Grandeza	n	Mín (ms)	Média (ms)	p95 (ms)	p99 (ms)	Máx (ms)
HARD-WARE	fim a fim, todas as sessões de <code>latencia_bancada.csv</code>	103	0,1	25,9	33,0	46,3	47,5

As linhas por modo vêm de `latencias.csv`, que fica fora do git por tamanho (09, 2026-08-26) e só tem as execuções exemplares; a linha da pior execução vem de `execucoes.csv`, versionado, e cobre todas.

6.1 `metrica_latencia`, no banco

Ambiente	n	Mín (ms)	Média (ms)	p95 (ms)	p99 (ms)	Máx (ms)
HARDWARE	102	0,000	25,667	32,000	43,000	46,000

O lote não grava `metrica_latencia` (a simulação fica nos CSV); as linhas de `HARDWARE` são as amostras de H3 que o backend gravou com a bancada no ar.

7. Casos de falha observados e corrigidos

Lidos de `context/09` (tabela “Decisões tomadas”), em ordem de data. O texto é o do registro, sem resumo: o que quebrou, o que foi feito e por quê.

7.1 I4 verificado por transição, não por par (2026-08-24)

Registro: I4 verificado por transição, não por par (Bloco 2)

O que foi feito: `core/seguranca.verificar_transicao()` passa a checar I4 sobre uma transição isolada. Um amarelo só pode suceder um verde, então `duracao_fase_anterior_s` de uma transição para `AMARELO` é a duração daquele verde.

Por quê: Furo encontrado por **teste de mutação**: com o verde mínimo sabotado, a violação não era acusada. A checagem antiga só rodava sobre pares consecutivos, e a primeira transição de cada execução ficava sem predecessor — justamente a mais exposta a um comando prematuro, logo após a partida do controlador. Registrado como teste de regressão.

7.2 `ESTENDER_VERDE` conta a partir de agora (2026-08-25)

Registro: `ESTENDER_VERDE` conta a partir de agora (defeito do Bloco 2, achado no Bloco 3)

O que foi feito: `core/priorizacao/fases.py` interpretava `duracao_s` como duração **total** do verde, contada do início dele; passa a contar **a partir do instante do comando**, com o teto de `verde_max` ainda ancorado no início (I5 preservado).

Por quê: O motor calcula `duracao_s = eta + margem`, que é tempo a partir de agora. Sob a leitura antiga, o comando virava seu oposto assim que o verde já durava mais que o pedido: pedir “segure mais 9 s para o VE passar” fechava o verde imediatamente. Efeito medido antes e depois, mesma seed e mesmo cenário: **6 paradas e 409 s de travessia → 0 parada e 313 s**. Nenhum teste unitário pegava — todos exercitavam extensões a partir de verdes recém-abertos. É a semântica de `PRE,<fase>,<dur_s>` do protocolo serial, então firmware e simulação voltam a concordar.

7.3 E7 passa a ser executada, e não só calculada (2026-08-25)

Registro: E7 passa a ser executada, e não só calculada (defeito do Bloco 2, achado no Bloco 3)

O que foi feito: O motor calculava PlanoCompensacao, guardava e **nada nunca o aplicava**. Passa a emitir ESTENDER_VERDE com o restante da duração planejada a cada fase que abre, enquanto a compensação vigora. O comando sai **sem id_veiculo**, e a máquina de estados só marca **em_preempcao** quando há VE associado.

Por quê: Os braços PREEMPCAO e PREEMPCAO_COMPENSADA saíam com resultados **idênticos até o último dígito** — E7 era um no-op e H2 não tinha mecanismo nenhum por trás — hipótese sem mecanismo não tem como ser testada. A distinção por **id_veiculo** importa porque **em_preempcao** viaja para **estado_semaforo_amostra**: sem ela, os dois ciclos de compensação seriam contabilizados como preempção e o custo transversal que H2 mede seria atribuído ao evento errado. Coberto por **test_compensacao_estende_de_fato_o_verde_das_fases**.

7.4 queue.xml sai do padrão; summary agregado a 60 s (2026-08-25)

Registro: queue.xml sai do padrão; summary agregado a 60 s (Bloco 3)

O que foi feito: O SUMO grava queue e summary a cada passo. Com passo de 0,1 s, a primeira execução completa (3.600 s) produziu **77 MB de queue.xml** e 10 MB de summary.xml. queue-output passa a ser opcional (**--saida-detalhada**) e summary passa a agregar a cada 60 s.

Por quê: Nas 600 execuções do Bloco 8 seriam ~46 GB só de fila, num disco de estudante — e para um dado **redundante**: os detectores E2 já medem fila com agregação de 300 s, e o coletor já acumula a fila máxima por aproximação em memória. É a mesma aritmética de P5 e da latência detalhada: volume bruto não é gratuito, e o que sustenta as hipóteses são os agregados. A saída bruta por execução caiu de ~88 MB para ~1,5 MB. Só apareceu ao rodar a primeira execução de 3.600 s de ponta a ponta — as de verificação, mais curtas, não davam a escala do problema.

7.5 Cada execução do lote limpa os CSV da própria pasta antes de rodar (2026-08-26)

Registro: Cada execução do lote limpa os CSV da própria pasta antes de rodar (defeito achado no piloto)

O que foi feito: **_limpar_csv_da_pasta()** apaga os quatro CSV da pasta da execução antes de executá-la.

Por quê: **Não é hipótese: aconteceu.** Quatro pontos tinham sido exercitados num teste curto (400 s) antes do piloto, e **gravar_csv()** **acrescenta** — comportamento certo para o arquivo consolidado, errado para a pasta de uma execução. O **execucoes.csv** do piloto saiu com **64 linhas para 60 execuções**, com as quatro sobras carregando a duração errada. Só apareceu porque a contagem foi conferida; a média não teria denunciado nada. Os quatro pontos foram reexecutados com **--repetir** e o descarte está em **descartes.csv**. Coberto por teste de regressão.

7.6 O pareamento da análise é por VE, não por execução (2026-08-26)

Registro: O pareamento da análise é por VE, não por execução

O que foi feito: **analysis/relatorio_piloto.py** compara os braços sobre a **interseção dos ids de VE presentes nos três**, por (cenário, seed), e declara no relatório quantos VEs a interseção descartou.

Por quê: Comparar a média de execução tem um viés silencioso e **conservador**, que é o que o faz passar despercebido: o último VE parte perto do fim do horizonte e, no baseline — mais lento —, pode não chegar dentro dos 3.600 s. Ele sai da média do **FIX0** mas fica na da preempção; a média do controle melhora por exclusão justamente do caso difícil, e a redução medida encolhe. No piloto isso valeu de 0 a 5 VEs por cenário. Coberto por teste.

7.7 Removida a linha leve/PREEMPCAO/seed=42 de execucao_simulacao (2026-08-26)

Registro: Removida a linha leve/PREEMPCAO/seed=42 de execucao_simulacao

O que foi feito: Sobra de uma execução de verificação do Bloco 3 (700 s, versão c8cc3c7, 477 transições), apagada com as transições em cascata. O banco fica com **60 execuções e 12 exemplares**, um por par cenário x modo.

Por quê: A linha estava marcada **exemplar**, o que dava **dois** exemplares para o par leve/PREEMPCAO e violava a decisão P5 — a regra que limita o volume de **estado_semaforo_amostra** e define quais execuções alimentam as figuras do capítulo 5. Não é dado experimental: não pertence a nenhuma matriz (a seed 42 está fora de 1..50), foi produzida por versão anterior do código e com duração fora do protocolo, e não aparece em nenhum CSV de **analysis/data/**. Apagá-la não altera número nenhum do piloto; deixá-la faria o Bloco 8 escolher entre dois exemplares para o mesmo par.

7.8 P16 resolvida pelo mecanismo, sem tocar em H1 (2026-08-31)

Registro: P16 resolvida pelo mecanismo, sem tocar em H1

O que foi feito: E3 passou a somar à janela de ativação o tempo de dissipação da fila do acesso de entrada, com teto derivado de **preempcao_timeout_s**. Medido nas mesmas seeds do piloto: **intenso 18,1% → 31,2%**, pior seed 13,0% → 27,9%, paradas do VE 2,76 → 0,08. A contingência de reformular H1 **não foi acionada**. Custo: a espera transversal no **intenso** subiu de +24,6% para +43,6%.

Por quê: A ordem — tentar o mecanismo antes de mexer na hipótese — foi declarada e **commitada antes do código** (1ae762e), e é o que torna o resultado defensável em vez de oportunista. A correção não introduz parâmetro livre: fila dos detectores E2, faixas da geometria da rede, headway de saturação medido, teto derivado. Os baselines **FIX0** idênticos aos do piloto provam que a melhora não vem de tráfego mais fácil. O custo transversal era um dos três resultados **declarados antes de medir** e vai para o texto como trade-off, não como nota de rodapé.

7.9 Defeito de posição do VE dentro do cruzamento, corrigido (2026-09-10)

Registro: Defeito de posição do VE dentro do cruzamento, corrigido (achado pela entrega 10.1)

O que foi feito: O adaptador somava **VAR_ROUTE_INDEX** e **VAR_LANEPOSITION** sem tratar as **faixas internas** do cruzamento, onde o primeiro ainda aponta para a via já deixada e o segundo recomeça do zero. O VE aparecia no começo da via anterior, ~490 m atrás, por ~1 s a cada travessia. Passa a assinar **VAR_ROAD_ID** e, em faixa interna, trata o VE como estando no fim da via atual. Regressão **sumo** que exige progresso monotônico ao longo da rota.

Por quê: Afeta **E1 e E2 em todos os cenários e nos três braços**, não só a contagem de conflitos: nesse intervalo o ETA saltava de 32 s para 63 s e o VE deixava de ser visto no cruzamento que de fato se aproximava. Só apareceu porque a instrumentação da 10.1 tornou observável uma disputa que sumia e voltava — argumento concreto a favor de medir antes de modelar. **Consequência:** os números de P16 vieram de código com o defeito; os do capítulo 5 virão do Bloco 8, com o código corrigido.

7.10 Firmware do UNO reescrito preservando o comportamento (2026-10-05)

Registro: Firmware do UNO reescrito preservando o comportamento (requisitos da entrega 5.3)

O que foi feito: Mantém a entrada, as prioridades (ambulância 1 · bombeiro 2 · polícia 3), as durações (9 · 8 · 7 s), a fila de um lugar e as mensagens do LCD. Muda: (1) transição segura em toda troca (verde mínimo, amarelo, all-red; nunca amarelo → verde); (2) o verde do VE conta a partir do verde exclusivo estabelecido; (3) **mesmo VE relendo a mesma rua renova** o verde, em vez de entrar na fila; (4) **tipo desconhecido é recusado**, em vez de virar prioridade 3; (5) fim da emergência volta ao ciclo **pelo eixo oposto**; (6) teto de 30 s de emergência contínua; (7) boot em all-red; (8) sem **String**; (9) quem perde o lugar na fila sai com **DESCARTADO**, em vez de sumir sem rastro.

Por quê: (1) é invariante de segurança, e o sketch o viola (verde de emergência aceso sem amarelo nem all-red). (2) garante ao VE o tempo que o tipo promete. (3) evita um segundo verde para o mesmo veículo. (4) evita que um texto corrompido na serial vire uma viatura. (5) atende primeiro quem esperou. (6) substitui o watchdog (I6, redefinido em 01 §6) e cumpre I5 na bancada sem o motor. Os itens (3) a (6) mudam o comportamento do sketch, e a equipe de hardware deve conferi-los.

7.11 Achado da primeira captura na bancada, antes de gravar o firmware novo (2026-10-06)

Registro: Achado da primeira captura na bancada, antes de gravar o firmware novo

O que foi feito: Com o firmware que estava na placa (o do repositório, de 2026-10-05, gravado pela equipe; a diferença no boot que parecia indicar outra versão é o bloqueio do `lcd.init()`) e o receptor ainda no RX (0), ~10 passagens do carrinho: decisões corretas (preempção, renovação, fila, descarte, teto de 30 s), e **dois EV, RECUSADO**, cada um ~3 s antes de um `PREEMP_INI`.

Por quê: O UNO não ecoa a linha recusada, então a causa não é observável daqui (linha do receptor corrompida, contato do fio, nível de 3,3 V no limite?). As 100 passagens do item 4 de 06 §6 medem a taxa, já com o receptor no A0; **RECUSADO** conta como falha do RNF05.

7.12 Gravação do UNO por python -m bridge.gravar_uno, e não pelo arduino-cli upload (2026-10-06)

Registro: Gravação do UNO por python -m bridge.gravar_uno, e não pelo arduino-cli upload (achado na bancada)

O que foi feito: Neste notebook, toda gravação pelo avrdude (8.0 e 6.3) sai corrompida: em cada página de 128 bytes, os bytes 60 a 63 ficam errados, e o programa trava no início (semáforos apagados, serial muda). O `arduino-cli upload` não verifica por padrão e dava a gravação por boa. O gravador novo fala o mesmo protocolo do bootloader (STK500 v1), manda cada comando em pedaços de 16 bytes com pausa e relê a flash inteira: 0 byte diferente, e o firmware roda.

Por quê: O byte 60 dos dados é o primeiro do segundo pacote USB de 64 bytes (4 de cabeçalho + 128 de dados): a corrupção está na fronteira de pacote do conversor USB da placa (16U2 de placa compatível, número de série 2017-2-25), e não no chip. A placa, a fiação e o receptor foram descartados um a um. Gravação que não confere passa a ser erro.

7.13 Linha pela metade seguida de 100 ms de silêncio é descartada (2026-10-06)

Registro: Linha pela metade seguida de 100 ms de silêncio é descartada (pedido do Felipe, depois do diagnóstico na bancada)

O que foi feito: Em cada entrada, se um pedaço de linha fica mais de `LINHA_PARADA_MS = 100` sem o próximo byte, o UNO o joga fora antes de seguir. O dublê modela isso em `receber_bytes`, e a comparação com o firmware passou a ser por pedaços de bytes, com Hypothesis gerando pedaços e esperas na fronteira de 95 a 105 ms.

Por quê: Causa dos **RECUSADO**, vista com o receptor no USB do notebook: ao reiniciar, o ESP8266 deixa lixo sem `\n` no fio, e ele grudava na primeira linha de verdade (que o receptor tinha mandado limpa). Uma linha chega inteira em ~26 ms a 9600; 100 ms cobre o `loop()` segurado pela escrita no LCD (~45 ms).

7.14 O boot publica a ST do all-red; o roteiro exige all-red de pelo menos 1 s (2026-10-06)

Registro: O boot publica a ST do all-red; o roteiro exige all-red de pelo menos 1 s

O que foi feito: `Controlador::boot()` chama `assentar()`, e a primeira ST (RRRR) sai junto com o `EV,BOOT`. O `bridge.verificar` espera o primeiro verde e confere all-red ≥ 1 s, em vez de ≈ 1 s.

Por quê: Na placa o `lcd.init()` bloqueia ~1,1 s logo depois do boot: os vermelhos ficam acesos, mas a primeira ST só saía com o primeiro verde, aos 1.125 ms. Mais all-red é seguro; o invariante (I3) é o mínimo.

7.15 O `millis()` do dublê do UNO trunca, como o da placa (2026-10-07)

Registro: O `millis()` do dublê do UNO trunca, como o da placa (achado ao rodar a suíte da 10.6; correção autorizada pelo Felipe)

O que foi feito: `UnoSimulado.t_dispositivo_ms` passa de `round(t · 1000)` a `floor(round(t · 1000, 6))`. O exemplo achado fica como `@example` no teste de propriedade de I1 a I4, com dois testes do relógio.

Por quê: O Hypothesis achou um all-red de 999 ms ($63002 \rightarrow 64001$) num roteiro com instantes no meio do milissegundo. Por dentro o dublê cumpria 1,000 s; o `round` caía num empate em .5 que o ruído de ponto flutuante decidia para lados opostos nos dois carimbos, e `bridge.verificar` acusava I3. Falhava também na main. O firmware não muda, e a comparação com ele (só ms inteiros) dá o mesmo resultado.

7.16 Saída dos `python -m` em UTF-8 (2026-10-07)

Registro: Saída dos `python -m` em UTF-8 (defeito achado na auditoria de 2026-10-07)

O que foi feito: `adapters/terminal.saida_utf8()` reconfigura `stdout` e `stderr` para UTF-8, e todo `main()` de `analysis/`, `bridge/`, `sim/` e `db/` a chama na primeira linha (08 §3). O `reconfigure(errors="replace")` que só `analysis.relatorio_piloto` tinha passa a ser esta mesma chamada.

Por quê: O `--help` de `bridge.demo`, `analysis.resumo_bancada` e `analysis.treino_politica` caía com a saída redirecionada (cp1252 não tem $\rightarrow$, $\geq$ e λ), e o mesmo risco valia para qualquer relatório impresso. Uma correção só, num lugar só, coberta por teste que roda os 22 `--help` em cp1252 e confere a chamada em todo `main()`. Um terminal que espere outra codificação mostra acento trocado, mas nada cai.

7.17 `versao_do_codigo()` marca árvore suja (2026-10-07)

Registro: `versao_do_codigo()` marca árvore suja (pendência da 10.4, que valia antes do Bloco 8)

O que foi feito: A versão gravada em `execucao_simulacao`, em `execucoes.csv` e nos CSV da rotulagem e da calibração passa a ser o HEAD curto com `-suja` quando há arquivo modificado (inclusive no índice) ou arquivo novo não ignorado, fora de `analysis/data/`, `docs/`, `context/` e `*.md`. Uma função só, em `sim/controlador/executor.py`; a cópia de `sim/calibracao/fluxo_saturacao.py` passa a usá-la. O `git status` roda sem locks opcionais.

Por quê: Sem isso, uma execução com código fora do commit gravava a versão de um commit que não era o dela, e isso aconteceu duas vezes (10.4 e 10.7). `analysis/data/` fica de fora porque o próprio lote escreve nela no meio da rodada; documentação não muda resultado. Os locks ficam de fora porque o lote chama a função em vários processos ao mesmo tempo. Marcar, e não recusar, porque as rodadas de verificação rodam de propósito com a árvore modificada: o Bloco 8 sai de uma main limpa, e a coluna é a prova. A ponte (`bridge/latencia.py`) ainda lê só o HEAD: a bancada já foi medida e não roda mais experimento.

7.18 Rodada de RNF05 e H3 feita; duas leituras repostas (2026-10-07)

Registro: Rodada de RNF05 e H3 feita; duas leituras repostas (achado na medição)

O que foi feito: Sessão 2026-10-07T16:18:08Z, commit 5409d93, emissor no COM5. Duas linhas do emissor chegaram ilegíveis à ponte e ficaram fora dos dois CSV: a da 1ª passagem (lixo de boot do ESP, que reinicia quando a ponte abre o COM5, grudado na linha) e a da 60ª (rajada de 0xFF no USB do emissor). Nas duas o UNO preemptou a rua certa. Ambas foram repostas na mesma rua (a 1ª na hora; a 60ª com uma passagem extra no fim), para fechar 100 leituras gravadas, 25 por rua. Evidência: `analysis/data/ponte_bancada_2026-10-07T16-18-08Z.log`.

Por quê: É o limite declarado na linha anterior, e aconteceu: a falha foi no instrumento (USB do emissor ao notebook), não no caminho do sistema. Contá-las como falha do RNF05 puniria o sistema por um defeito do aparato; contá-las como sucesso exigiria um número tirado do log, à mão. Repor mantém o denominador inteiro em dado gravado.

7.19 Carimbo atrasado: as amostras ficam, e o resumo ganha uma linha de sensibilidade (2026-10-07)

Registro: Carimbo atrasado: as amostras ficam, e o resumo ganha uma linha de sensibilidade (decisão do Felipe)

O que foi feito: Uma amostra tem carimbo atrasado quando havia ≥ 10 bytes já esperando na porta, em qualquer lado (BYTES_CARIMBO_ATRASADO). O resultado de H3 usa as 100 amostras; o `resumo_bancada` acrescenta a mesma tabela sem as atrasadas. Na rodada foram 2: 0,114 ms (35 bytes no emissor, 86 no UNO) e 8,8 ms (15 no emissor), as duas abaixo do piso físico de ~ 23 ms.

Por quê: O limiar foi escolhido **depois** de ver os dados, e isso fica declarado: a distribuição é bimodal (≤ 6 bytes em 98 amostras, 15 e 35 nas outras duas), então qualquer limiar de 7 a 14 separa as mesmas amostras. Excluir sem mostrar seria escolher o dado; mostrar as duas linhas deixa a leitura com quem lê. O veredito não muda: p95 31,6 ms com elas, 33,0 ms sem. Achado relacionado: em 95 de 100 amostras havia 6 bytes esperando na porta do UNO quando a decisão foi carimbada, provavelmente lote do conversor USB da placa. Isso atrasa `t_atuacao` em até ~ 6 ms e infla a latência medida, contra a hipótese.

7.20 Checklist: dois ajustes no script depois da rodada, sem mudar limite (2026-10-07)

Registro: Checklist: dois ajustes no script depois da rodada, sem mudar limite (achado ao ler o relatório)

O que foi feito: (1) Itens 1 e 12 dão “sem veredito”, e não “não atende”, quando a sessão é curta demais e não mostra falha. (2) O item 15 mede até a ST trazer a lista **inteira** do primeiro envio da ponte depois do `BOOT`, e não a primeira lista diferente de 000. Os limites (12 s $\pm$ 60 ms, 5 min, 30 min, 2 s) não mudaram, e o CSV é o mesmo.

Por quê: (1) As três sessões de 20 s do item 15 apareciam como falha dos itens 1 e 12, o que o dado não mostra. (2) O UNO aplica as três linhas AUT uma por vez e publica uma ST a cada uma (200, depois 210, no mesmo `millis()`). A primeira lista não nula ainda não é a da Central. No dado, a diferença foi de ~ 20 ms.

7.21 Item 5 do checklist feito, com dois desvios do roteiro declarado (2026-10-07)

Registro: Item 5 do checklist feito, com dois desvios do roteiro declarado (rodada do Felipe; registro em vez de repetição, decisão do Felipe)

O que foi feito: Sessão 2026-10-08T00:32:44Z (21:32 no horário local de 2026-10-07), `python -m analysis.checklist_bancada --item5 (analysis/data/checklist_bancada_item5_2026-10-07.md)`: janela de **53,9 s**, da ambulância em ocorrência até o controle; 0 leitura do emissor e 0 evento do UNO nela; 135 ST em ciclo; 4 ciclos inteiros de 12.000 ms; o controle na tag da Rua 1 deu `PREEMP_INI`. O critério de 06 §6, declarado antes (`bc88ecd`), é atendido. **Desvios do roteiro:** a tag passou **4 ou 5 vezes** (contagem do Felipe), e não 10, e a janela teve 53,9 s, e não pelo menos 1 min. LCD parado em todas as passagens (observação do Felipe).

Por quê: Os desvios não alteram o que o critério mede: nenhuma passagem gerou envio, e a janela cobre 4 ciclos inteiros. Mas a amostra é menor que a declarada, e isso fica escrito como foi. A premissa de que o RC522 leu a tag nova (MIFARE crua, sem configuração) não é verificável sem mudar o sketch: é observação.

7.22 A abertura do COM5 reinicia o emissor, e o lixo do boot suja a primeira linha da sessão (2026-10-07)

Registro: A abertura do COM5 reinicia o emissor, e o lixo do boot suja a primeira linha da sessão (achado no item 5)

O que foi feito: As duas primeiras rodadas do item 5 (sessões 00:25:27Z e 00:28:01Z) saíram “sem veredito”: a linha do controle chegou com lixo na frente (`\x00...\x92...` e `\x92\xfd`) e a ponte a descartou, embora o UNO tenha preemptado a Rua 1. Escuta crua do COM5 a 74880 baud (`python -m bridge.escuta_emissor, analysis/data/emissor_74880_2026-10-07.log`): `rst cause:2` só na abertura da porta. Três passagens da tag nova não produziram nenhum byte, e a da Rua 1 produziu os 109 bytes da sua linha. Daí em diante, a rodada do item 5 começa com uma passagem de **aquecimento** numa tag do mapa, antes de abrir a ocorrência (vira `SEM_OCORRENCIA`, fora da janela).

Por quê: É o mesmo “lixo de boot do ESP” que tirou a 1ª passagem da rodada do RNF05 (linha anterior de 2026-10-07): o boot sai a 74880, sem `\n` legível a 9600, e gruda na primeira linha do sketch. O roteiro do item 5 punha justamente o controle como primeira linha. A hipótese alternativa (o emissor reiniciar ao ler a tag fora do mapa) foi descartada pela escuta. As duas sessões ficam no CSV como evidência, com o log da ponte de cada uma.

7.23 Bloco 8 rodado: 650 execuções, 0 descarte (2026-10-08)

Registro: Bloco 8 rodado: 650 execuções, 0 descarte (execução; uma interrupção, reexecutada do zero)

O que foi feito: analysis/data/bloco8/: execucoes.csv (650 linhas, todas 85b1803, sem -suja), ve_por_execucao.csv, transversal_por_execucao.csv, conflitos_por_execucao.csv, descartes.csv e o log do lote (lote_bloco8.log); latencias.csv fica fora do git, como os demais. 650 linhas em execucao_simulacao, todas finalizadas, 13 exemplares (P5). Rodou num worktree limpo da main (dados/bloco8), com --paralelo 6, em 3h36, desligado da sessão do Claude. **Interrupção:** a primeira corrida (2026-10-07, ~22:10) foi parada em 28 de 650 porque o computador precisou reiniciar; os CSV ainda não tinham sido consolidados, e as 36 linhas que ela deixou no banco foram apagadas pelo --repetir da corrida nova, que as registrou em descartes.csv como REEXECUCAO, com o motivo. Antes dela, um disparo de segundos foi abortado sem gravar nada, porque o log estava na raiz do worktree e marcaria a versão como -suja.

Por quê: Todos os 650 pontos saíram da mesma corrida e do mesmo commit. Reexecutar do zero, em vez de completar a corrida interrompida, evita juntar execuções de duas corridas e mantém a consolidação em ordem determinística. O lote não tem retomada no meio, porque consolida no fim, e o --repetir é o caminho documentado para remover evidência anterior. Nenhum número das hipóteses é lido aqui: a análise é o Bloco 9 e a 10.8.

7.24 Um carrinho por tipo: a bancada ganha os emissores do bombeiro e da polícia (2026-10-08)

Registro: Um carrinho por tipo: a bancada ganha os emissores do bombeiro e da polícia (decisão do Felipe, com o hardware que chegou)

O que foi feito: Dois NodeMCUs com RC522, com a mesma pinagem do emissor da ambulância, levam o sketch dela com **uma** linha trocada, a do tipo (firmware/nodemcu/veiculo_bombeiro/ e veiculo_policia/); tests/firmware/test_sketches_nodemcu.py confere que nada mais difere. Gravados com o arduino-cli (esp8266 3.1.2, MFR522 1.4.12). Receptor, UNO, protocolo, ponte e regra não mudam: o UNO já aceitava os três tipos (05 §3.2). Os seeds ganham EMISSOR_VE_02 e EMISSOR_VE_03 (03 §5), sem migration. O passo 5 da demonstração (05 §7) passa a usar os carrinhos (pedido do Felipe): ambulância × bombeiro e ambulância × polícia, com a injeção como plano B (--sem-carrinho).

Por quê: O UNO e o dublê sempre trataram os três tipos; faltava só o veículo físico, e a injeção era a única forma de mostrar bombeiro e polícia. Copiar o sketch, e não parametrizá-lo, preserva a decisão de 2026-10-05 (os sketches da equipe não mudam) e mantém o da ambulância, o de H3, intacto. No carrinho da polícia o RST do RC522 vai ao 3V3, porque o pino D3 da placa não leva o nível ao fio (diagnóstico e decisão em 05 §1); o código continua o mesmo. **H3 não é refeita:** foi medida com a ambulância, e o caminho dos outros dois é o mesmo, com 2 ou 3 caracteres a menos na linha do receptor (05 §4.3).

7.25 Item 15 do checklist: lista da Central 000 no BOOT sai “sem veredito” (2026-10-08)

Registro: Item 15 do checklist: lista da Central 000 no BOOT sai “sem veredito” (achado ao gerar o relatório de validação; ajuste depois dos dados, decisão do Felipe)

O que foi feito: Quando a ponte não escreve a lista em 2 s do BOOT, analysis.checklist_bancada consulta com --banco a lista da Central naquele instante (ocorrências abertas de veículos ativos, a regra de criticidade_por_tipo). 000: “sem veredito”, nada a reenviar; outra lista: “não atende”. Sem o banco, “sem veredito”. O limite de 2 s não muda, e o CSV é o mesmo.

Por quê: Rodado com --todas, o item saía “não atende” em 4 sessões: as três do item 5 (00:25:27Z, 00:28:01Z, 00:32:44Z) e a dos três carrinhos (2026-10-08T20:40:56Z), com a primeira lista escrita 10,46 s, 9,26 s, 22,10 s e 140,66 s depois do BOOT. Em nenhuma havia ocorrência aberta no BOOT: nas do item 5 a lista foi posta à mão por PUT /autorizacoes (o roteiro permite, 06 §6), e na dos carrinhos a primeira ocorrência abriu às 20:43:18Z. O que o script cronometrava era a abertura da ocorrência, não a volta da lista depois do reinício, que é o que o item pergunta. **É o mesmo tipo de ajuste da linha “Checklist: dois ajustes no script” de 2026-10-07, e fica declarado que veio depois de ver os dados.** Nas sessões em que a lista voltou, nada muda.

7.26 RF03 não é cumprido ao pé da letra no Bloco 8 (2026-10-08)

Registro: RF03 não é cumprido ao pé da letra no Bloco 8 (achado ao gerar o relatório de validação; decisão do Felipe)

O que foi feito: O critério de 06 §2 é `waitingCount == 0` para o VE. Em `analysis/data/bloco8/ve_por_execucao.csv`, no braço `PREEMPCAO`, passam sem nenhuma parada 185 de 298 VEs no `leve` (62,1%), 210 de 262 no `moderado` (80,2%), 198 de 250 no `intenso` (79,2%) e 289 de 566 no `multiplas_emergencias` (51,1%) (os mesmos números da T4 adaptada). No `FIXO`, nenhum VE passa sem parar. O relatório de validação dá **RF03 FALHOU**. `tests/e2e/test_corredor_verde.py` confere o critério literal sobre o CSV e fica `xfail estrito`: se um dia passar, a suíte quebra. Dois testes sem `xfail` fixam só a direção (paradas por VE abaixo do `FIXO` em todo cenário e braço).

Por quê: Um teste de uma seed só passaria ou falharia conforme a seed, e escolher uma que passa seria escolher o resultado. As paradas são curtas: tempo de espera mediano de quem parou de 2,3 s no `leve` e no `moderado` e 9,2 s no `intenso`, contra 68,5 a 109,5 s de mediana no `FIXO` (as medianas saem do relatório de validação, `_evidencia_rf03`). **Onde** os VEs param não está nos dados do lote (paradas é uma contagem); o traço gravado (`intenso`, seed 1) não tem nenhuma parada. Reformular o critério depois dos dados seria ajustar a régua ao resultado: fica como está, e vai para a discussão do capítulo 5 e para a mensagem ao orientador.

7.27 Quatro arquivos de teste de 06 §2 não existiam (2026-10-08)

Registro: Quatro arquivos de teste de 06 §2 não existiam (achado ao gerar o relatório de validação; decisão do Felipe: escrever os quatro)

O que foi feito: A tabela de rastreabilidade citava `test_e2e_preempcao.py` (RF02 e H3), `test_corredor_verde.py` (RF03), `test_recalculo.py` (RF07) e `test_soak.py` (RNF02), e nenhum existia; `tests/e2e/` só tinha um `__init__.py` vazio. Escritos em 2026-10-08: `backend/tests/core/test_recalculo.py` (bifurcação artificial: a rota nova troca os cruzamentos-alvo, o motor solta o que saiu da rota e preempta o novo); `tests/e2e/test_e2e_preempcao.py` (a cadeia carrinho → receptor → UNO → ponte contra o duplê, e os critérios de RF02 e H3 sobre o `latencia_bancada.csv` versionado); `tests/e2e/test_corredor_verde.py` (linha anterior); `sim/tests/test_soak.py` (3.600 s simulados com o laço completo do executor e o RSS do processo amostrado numa thread, critério de $\pm 10\%$ declarado no teste antes da primeira execução; marcas `sumo` e `lento`). O `tests/e2e/__init__.py` saiu: com ele, o pacote `tests` da raiz colidiria com o `tests` de `backend/`, como nas outras pastas de `tests/`.

Por quê: Sem os arquivos, o relatório daria “SEM TESTE” para RF07 e RNF02, que não tinham evidência nenhuma, e só evidência de bancada para RF02 e H3. O RNF02 mede voltas do laço, e não relógio: 3.600 s simulados são as 36.000 decisões de 60 min de operação; a operação contínua no relógio está no item 12 da bancada. Os números do duplê não são dado experimental.

8. Checklist de aceitação do protótipo físico

Os itens de `context/06 §6`, lidos do arquivo, com o registro de cada um (data, executor e resultado). Na última coluna, o que `analysis.checklist_bancada` diz de cada item em **todas** as sessões gravadas, com o banco (itens 9 e 15). Os itens 4, 7 e 7b são julgados por `analysis.resumo_bancada` (seção 4); 6, 8, 13 e 14 são observação.

#	Verificação e registro	OK	Pelo dado, por sessão
1	Ciclo alterna os 2 eixos (ciclo de 12 s) por 5 min sem travar; liga em all-red. 2026-10-07, Felipe, sessão 17:44:10Z (<code>analysis/data/checklist_bancada_2026-10-07.md</code>): primeiro verde aos 1.125 ms, depois do all-red; 157 ciclos puros, de 12.000 ou 12.001 ms, exceto um de 12.032 ms (o do <code>SEM_OCORRENCIA</code> do 5b, provavelmente o aviso no LCD por I2C segurando o <code>loop()</code>); 1.140 s contínuos de ciclo	OK	atende: 2, sem veredito: 7
2	Nunca há verde nos dois eixos ao mesmo tempo; em emergência só a aproximação do VE fica verde — 5 min de observação e a telemetria do mesmo período. 2026-10-07, Felipe, sessão 17:44:10Z : 0 de 4.948 ST com verde nos dois eixos, nenhum verde novo fora da aproximação do VE, verde exclusivo nas 5 emergências atendidas até o fim; observação do Felipe nos 6 min de ciclo puro	OK	atende: 9

#	Verificação e registro	OK	Pelo dado, por sessão
3	Toda transição verde→vermelho passa por amarelo, e há all-red antes de todo verde novo, inclusive na entrada e na saída da emergência. 2026-10-07, sessão 17: 44:10Z: 972 mudanças de luz, 4 entradas e 3 saídas de emergência (a quarta foi cortada pelo item 14), 0 violações de I2, I3 ou I4	OK	atende: 6, sem veredito: 3
4	100 passagens sobre as tags das ruas: em ≥ 95 a linha chega ao UNO com a rua certa, ou seja, há um evento de decisão com a rua da tag — RNF05. Um RECUSADO conta como falha (a linha chegou corrompida). Medido na mesma rodada do 7b, com o emissor no USB do notebook (decisão de 2026-10-06): o denominador são as leituras que o emissor imprimiu, e o resultado é declarado nessa condição, que alimenta o emissor melhor que a bateria de 9 V da demonstração. Cada leitura vira uma linha de <code>analysis/data/deteccoes_bancada.csv</code> ; a taxa sai de <code>python -m analysis.resumo_bancada</code> . 2026-10-07: 100 de 100 (25 por rua), todas PREEMP_INI; duas linhas do emissor que chegaram ilegíveis ao notebook ficaram fora do denominador e foram repostas (09, 2026-10-07)	OK	atende (sessão das 100 passagens, <code>resumo_bancada</code>)
5	Tag fora das 4 ruas não gera envio nem mexe no semáforo. Roteiro (declarado em 2026-10-07, antes da rodada): ponte com <code>--porta COM3 --porta-veiculo COM5 --telemetria</code> , emissor no USB; a ambulância recebe ocorrência (pela Central, ou com o compose parado por <code>PUT /autorizacoes</code> na ponte); com o ciclo correndo, a tag fora do mapa passa pelo leitor do carrinho 10 vezes , com pelo menos 4 s entre uma e outra, ao longo de pelo menos 1 min; por último, uma passagem de controle na tag da Rua 1. Julgado por <code>python -m analysis.checklist_bancada --item5</code> ; o LCD sem mudança nas 10 passagens e a contagem delas são observação. 2026-10-07, Felipe, sessão 00:32:44Z (<code>analysis/data/checklist_bancada_item5_2026-10-07.md</code>), depois de uma passagem de aquecimento na Rua 1 antes da ocorrência (o lixo de boot do emissor suja a primeira linha da sessão, 09): janela de 53,9 s sem leitura do emissor nem evento do UNO, 4 ciclos de 12.000 ms, controle PREEMP_INI. Desvios do roteiro, registrados em 09: 4 ou 5 passagens da tag nova (não 10) e janela abaixo de 1 min. LCD parado em todas (observação do Felipe)	OK	atende: 1, sem veredito: 2
5b	Tag sem ocorrência (volta em 2026-10-06): sem ocorrência aberta para a ambulância na Central, a passagem gera SEM_OCORRENCIA, não mexe no semáforo, e o LCD mostra SEM_OCORRENCIA por 3 s; abrindo a ocorrência, a passagem seguinte preempta. 2026-10-07, Felipe, sessão 17:44:10Z: SEM_OCORRENCIA com a ambulância em 0 e o semáforo como estava; reaberta com criticidade 2, a passagem seguinte deu PREEMP_INI. LCD SEM_OCORRENCIA por ~3 s (observação do Felipe)	OK	atende: 4, sem veredito: 5
6	LCD mostra o VE e a rua em < 1 s após a leitura. 2026-10-07, Felipe: observação, sem atraso perceptível nas passagens do 5b e do 10; sem medida instrumentada	OK	—
7	Preempção iniciada (PREEMP_INI) em < 3 s da leitura da tag — RF02. 2026-10-07: nas 100 passagens do 7b, máximo de 46,3 ms	OK	atende (sessão das 100 passagens, <code>resumo_bancada</code>)

#	Verificação e registro	OK	Pelo dado, por sessão
7b	H3 — p95 de <code>latencia_total_ms</code> < 200 ms em 100 passagens (as do item 4), com o emissor no USB do notebook e a ambulância em serviço na Central; mín/mediana/máx registrados. <code>python -m bridge.main --porta COM3 --porta-veiculo <COM do emissor></code> (COM5 no notebook do Felipe, CP2102); cada passagem atendida vira uma linha de <code>analysis/data/latencia_bancada.csv</code> (05 §6); n, mín, mediana, p95 e máx saem de <code>python -m analysis.resumo_bancada</code> . 2026-10-07: p95 31,6 ms (n = 100; sem as 2 de carimbo atrasado, n = 98 e p95 33,0 ms), <code>analysis/data/resumo_bancada_2026-10-07.md</code>	OK	atende (sessão das 100 passagens, <code>resumo_bancada</code>)
8	Dashboard mostra o evento em tempo real. 2026-10-07, Felipe: observação, o ciclo ao vivo e o <code>PREEMP_INI</code> do 5b na hora	OK	—
9	Log gravado no PostgreSQL com <code>id_correlacao</code> completo. 2026-10-07, sessão 17: 44:10Z (<code>--banco</code>): os 14 eventos de decisão com exatamente uma linha em <code>log_prioridade</code> no mesmo carimbo, todas com <code>id_correlacao</code> ; os 6 <code>PREEMP_INI</code> fechados; as 2 amostras de H3 em <code>metrica_latencia</code> com o mesmo <code>id_correlacao</code> do log	OK	atende: 3, sem veredito: 6
10	Com a bancada montada, a injeção pela ponte mostra a regra de prioridade pela criticidade: o VE mais crítico interrompe o outro pelo amarelo e all-red, o interrompido vai para a fila (LCD Fila:...) e é atendido depois. 2026-10-07, Felipe, sessão 17:44:10Z: ambulância (2) pelo carrinho na Rua 3, bombeiro (1) injetado na Rua 1 3 s depois: <code>PREEMP_INI</code> dele e <code>FILA</code> dela no mesmo <code>millis()</code> , e ela atendida no <code>PREEMP_FIM</code> dele. LCD Fila:AMBU na R3 (observação do Felipe)	OK	atende: 3, sem veredito: 6
11	Renovações sucessivas param no teto de 30 s (<code>EV, TIMEOUT</code>) e o ciclo volta pelo eixo oposto. 2026-10-07, sessão 17:44:10Z: ambulância injetada na Rua 4 com 6 renovações; <code>TIMEOUT</code> 30.000 ms depois do <code>PREEMP_INI</code> , e o ciclo voltou por <code>GRRR</code>	OK	atende: 2, sem veredito: 7
12	Operação contínua de 30 min sem travamento ou reboot. 2026-10-07, sessão 17: 44:10Z: 33,1 min da primeira à última <code>ST</code> , nenhum reinício do <code>UNO</code> , nenhuma linha ilegível, nenhuma reconexão; maior intervalo entre duas <code>ST</code> de 1,12 s, que é o <code>lcd.init()</code> do boot (depois dele, 0,53 s)	OK	atende: 1, sem veredito: 8
13	Nenhum LED com brilho anômalo ou aquecimento perceptível. 2026-10-07, Felipe: observação durante os 33 min do item 12	OK	—
14	Ponte encerrada no meio de uma emergência → o semáforo segue, e o carrinho continua preemptando com a última lista da Central. 2026-10-07, Felipe: ponte encerrada 2 s depois do <code>PREEMP_INI</code> na Rua 2 (última <code>ST RRR</code> , regime E, sessão 17: 44:10Z); com ela fechada, o verde terminou, o ciclo voltou, e o carrinho na Rua 4 preemptou (observação do Felipe)	OK	sem veredito: 9
15	Ponte reiniciada (o <code>UNO</code> reinicia junto) → o <code>UNO</code> volta negando todos, e com o backend no ar a lista da Central volta em até ~1 s. 2026-10-07, 4 aberturas da porta (a do item 12 e três sessões de 20 s): primeira <code>ST</code> sempre 000; a lista inteira na <code>ST</code> 1,17 a 1,20 s depois do <code>BOOT</code> . O tempo é o do <code>lcd.init()</code> : a ponte escreve a lista 0,11 a 0,23 s depois do <code>BOOT</code> , e o <code>UNO</code> a aplica no <code>millis()</code> 1.125	OK	atende: 5, sem veredito: 4

8.1 Veredito por sessão

Sessão (UTC)	1	2	3	5	5b	9	10	11	12	14	15
2026-10-07 17:44:10Z	atende	atende	atende	não julgado	atende	atende	atende	atende	atende	sem veredito	atende
2026-10-07 18:18:37Z	sem veredito	atende	sem veredito	não julgado	sem veredito	sem veredito	sem veredito	sem veredito	sem veredito	sem veredito	atende
2026-10-07 18:19:00Z	sem veredito	atende	sem veredito	não julgado	sem veredito	sem veredito	sem veredito	sem veredito	sem veredito	sem veredito	atende
2026-10-07 18:19:23Z	sem veredito	atende	sem veredito	não julgado	sem veredito	sem veredito	sem veredito	sem veredito	sem veredito	sem veredito	atende
2026-10-07 18:40:29Z	sem veredito	atende	atende	não julgado	atende	atende	atende	sem veredito	sem veredito	sem veredito	atende
2026-10-08 00:25:27Z	sem veredito	atende	atende	sem veredito	sem veredito	sem veredito	sem veredito	sem veredito	sem veredito	sem veredito	sem veredito
2026-10-08 00:28:01Z	sem veredito	atende	atende	sem veredito	sem veredito	sem veredito	sem veredito	sem veredito	sem veredito	sem veredito	sem veredito
2026-10-08 00:32:44Z	sem veredito	atende	atende	atende	atende	sem veredito	sem veredito	sem veredito	sem veredito	sem veredito	sem veredito
2026-10-08 20:40:56Z	atende	atende	atende	não julgado	atende	atende	atende	atende	sem veredito	sem veredito	sem veredito

“Não julgado”: o item 5 só é julgado nas rodadas dele, e o 9 só com o banco.

8.2 Assinaturas

Declaramos que o checklist acima foi executado nas datas registradas em cada item.

- Felipe Rafael Tancredi Pascucci (T895HG3): _____ Data: ____/____/____
- Giovanna Santos da Silva (G828HA5): _____ Data: ____/____/____
- Isabelle Rosa Moura Ferreira (N075465): _____ Data: ____/____/____

9. Anexos

Logs e relatórios gravados, versionados no repositório (sha256, 16 primeiros):

Arquivo	Tamanho	sha256
analysis/data/bloco8/lote_bloco8.log	731.9 KiB	32a3a9aea1299ca9
analysis/data/checklist_bancada_2026-10-07.md	12.5 KiB	4fc9adfd27e9748
analysis/data/checklist_bancada_item5_2026-10-07.md	3.8 KiB	e0e174e0efae3fe1

Arquivo	Tamanho	sha256
analysis/data/emissor_74880_2026-10-07.log	0.6 KiB	6899bdb09c04b2c1
analysis/data/ponte_bancada_2026-10-07T16-18-08Z.log	645.6 KiB	21ca8334924bf9d2
analysis/data/ponte_bancada_2026-10-07T17-44-10Z.log	585.6 KiB	8ef4f2a1e48bf482
analysis/data/ponte_bancada_2026-10-08T00-25-27Z.log	2.0 KiB	c67c4c059df57d0e
analysis/data/ponte_bancada_2026-10-08T00-28-01Z.log	1.5 KiB	61d17c2502edee3b
analysis/data/ponte_bancada_2026-10-08T00-32-44Z.log	1.7 KiB	ea232a40e39c56ed
analysis/data/resumo_bancada_2026-10-07.md	1.1 KiB	6c2763c608d874f1

A anexar, com a maquete pronta:

- fotos da bancada montada (os três carrinhos, o cruzamento e o LCD);
- capturas do dashboard durante uma simulação e durante a bancada;
- vídeo de backup da demonstração (`context/08 §6`).